Especificação SDD — TaskAnalyzer (Bootcamp III)

1. Visão geral e contrato de negócio

1.1 Identificação

- **Nome completo:** Gabriel Gonçalves Sá
- **Curso:** Ciência da Computação — CEUB
- **RA:** 22352853
- **E-mail institucional:** gabriel.gsa@sempreceub.com

1.2 Propósito do módulo

O **TaskAnalyzer** analisa conjuntos de tarefas executadas e extrai indicadores de produtividade: tempo médio de conclusão (geral e por prioridade), taxa de atraso (tarefas concluídas além do prazo) e totalizadores por nível de prioridade (Baixa, Média, Alta).

1.3 Contrato executável de interface

Função pública: `analyze_tasks(tasks) -> dict`. Exceção pública: `TaskValidationError` (subclasse de `ValueError`).

1.3.1 Entradas

`tasks` é uma lista de tarefas. Cada tarefa é um dicionário com os campos:

Campo	Tipo	Descrição / restrição
`id`	int	Identificador único da tarefa.
`titulo`	str	Título ou descrição curta.
`prioridade`	str	`'Baixa'`, `'Média'` ou `'Alta'`.
`data_criacao`	datetime	Data e hora de inserção no sistema (aceita também string ISO 8601).
`prazo_limite`	datetime	Data limite para término sem atraso (aceita também string ISO 8601).
`data_conclusao`	datetime / None	Data efetiva de término; nula se não finalizada (aceita também string ISO 8601).
`status`	str	`'Pendente'` ou `'Concluída'`.

1.3.2 Saídas

`analyze_tasks` retorna um dicionário com as chaves:

Chave	Tipo	Descrição
`total_tarefas`	int	Total de tarefas analisadas.
`total_concluidas`	int	Tarefas com status `'Concluída'`.
`total_pendentes`	int	Tarefas com status `'Pendente'`.
`tempo_medio_conclusao_horas`	dict	Média em horas de `data_conclusao - data_criacao`: chaves `geral`, `Baixa`, `Média`, `Alta`.
`taxa_atraso_percentual`	dict	Percentual de concluídas com `data_conclusao > prazo_limite`: chaves `geral`, `Baixa`, `Média`, `Alta`.
`indicadores_prioridade`	dict	Contagem total de tarefas por prioridade: chaves `Baixa`, `Média`, `Alta`.

Valores percentuais e médias são arredondados em 2 casas decimais.

1.3.3 Regras de negócio e restrições

- Tempo médio e taxa de atraso consideram **estrita e exclusivamente** tarefas com status `'Concluída'`.
- Concluir exatamente no prazo (`data_conclusao == prazo_limite`) **não** é atraso.
- Validação temporal: `data_conclusao` (se presente) não pode ser anterior a `data_criacao`.
- Divisores são validados preventivamente: sem tarefas concluídas (no geral ou em uma prioridade), as médias e taxas daquela categoria valem `0.0`, sem `ZeroDivisionError`.
- Datas tratadas de forma consistente, preferencialmente em ISO 8601.

2. Cenários de aceite e Test Harness

2.1 Cenários (Dado — Quando — Então)

Cenário 1 — Sucesso na extração de métricas - **Dado** um conjunto de tarefas válidas, com prioridades Baixa, Média e Alta, prazos e status distintos (Pendente e Concluída), - **quando** o `analyze_tasks` for executado, - **então** as métricas devem ser precisas: `total_tarefas`, `total_concluidas`, `total_pendentes`, tempo médio de conclusão (geral e por prioridade), taxa de atraso (percentual exato) e contagem por prioridade.

Cenário 2 — Erro crítico de validação de dados - **Dado** uma entrada corrompida ou logicamente inválida (por exemplo, `data_conclusao` anterior a `data_criacao`), - **quando** o

`analyze_tasks` for executado, - **então** o sistema deve interromper a execução e lançar `TaskValidationError`, com mensagem clara, descritiva e orientativa.

Cenário 3 — Casos de borda (divisão por zero) - **Dado** uma entrada vazia, ou contendo apenas tarefas pendentes, ou sem tarefas concluídas em alguma prioridade, - **quando** o `analyze_tasks` for executado, - **então** as médias e taxas sem base de cálculo devem retornar `0.0`, sem lançar exceções não tratadas.

2.2 Test Harness

Os cenários são convertidos em testes `pytest` (`tests/test_harness.py`) com fixtures para massas perfeitas, vazias e com anomalias temporais. O código gerado por IA só é aceito com **100% de aprovação**.

3. Governança de contexto

As regras para agentes de IA estão em `CONTEXT_RULES.md` (Python 3.11+, type hints, Clean Code/PEP 8, docstrings Google style, `logging`, exceções granulares e proibições explícitas).

4. Arquitetura do repositório e homologação

```
sdd-task-analyzer/
├── README.md
├── CONTEXT_RULES.md
├── specs/
│   └── task_analyzer_spec.md
├── tests/
│   └── test_harness.py
└── src/
    └── task_analyzer.py
```

Gates de homologação humana: verificação funcional, auditoria de contrato, análise estática, avaliação de bordas e merge somente com aval humano.